Coming from superpowers?

You already know 80% of chris-code.

Same `brainstorm → plan → execute → review → finish` pipeline.
Same skill names. Same TDD and systematic-debugging discipline.

This deck is the other **20%**, and the *why*.

What feels different, day to day

1

Shorter specs & plans

No more 2,500-word specs or 10k-word plans full of code the implementer throws away.

2

Dedicated agents

Coding and review run through named agents auto-dispatched by file type. You rarely pick one.

3

Same gates, every task

Spec + quality reviews in parallel, triaged by script → a commit gate (lint + verify). Nothing is "too small to review."

Six thematic shifts

1. **Lean artifacts** over exhaustive ones
2. **A real agent layer**, dispatched by scope
3. **Review** is a uniform, multi-stage gate
4. **Parallelism** is a feature, not a footgun
5. **Native tools** first, with a hard gate
6. **Coherence enforcement** — every determined change fits the codebase *(the largest divergence)*

1 · Lean artifacts over exhaustive ones

superpowers

Spec written "as if zero context"

Plan: complete code in every step

specs grew past 2,500 w · plans 5,000–10,000 w

agents discarded the pasted code and rewrote it

→

chris-code

Spec = invariants only

Plan = what & where, no code

code written against the real repo

"contracts stay, choreography goes"

2 · A real agent layer, dispatched by scope

superpowers dispatches a generic `Task (general-purpose)` subagent steered by a prompt template. chris-code routes to dedicated agents:

a .py file in a torch project

↓ dispatched by scope (extension + deps) ↓

coder — exclusive	quality-reviewer — additive	review-lite — whole-change gate	design-reviewer — verification gate
pytorch-coder ✓	**python-quality-reviewer**	**python-review-lite**	**python-design-reviewer**
~~python-coder~~	**pytorch-quality-reviewer**		
most-specific wins	all matching fire	idiom + lint, whole diff	senior cohesion, read-only

3 • Review: kept the spine, hardened it

superpowers self-review → spec review → quality review → final review

chris-code spec + quality → + scripted triage → + commit gate → + full-diff review-lite

Same two reviews, now **dispatched together**, with tagged findings routed by script into at most two fix rounds (an S3+ contract-reach finding gets a decision doc first). chris-code makes the reviewers **dedicated, scope-dispatched agents**, adds a **mandatory lint gate** and a **spec-blind intent re-check**, and re-reads the actual code at every gate. More gates raise *recall*, not proof — the decorrelated axes (deterministic lint, spec-blind intent, mutation probe) carry the weight, not repetition.

4 · Parallelism is a feature, not a footgun

✗ superpowers — "dispatch parallel implementers" is a Red Flag

implementer A | implementer B | implementer C

unscheduled: they edit the same files and collide

✓ chris-code — map file footprints, then stage

Stage 1	A · api/	B · db/	C · ui/
Stage 2	D · api/ + db/ (waits — overlaps A & B)		

5 · Native tools first, with a hard gate



The split is only at "no native tool": superpowers falls back; chris-code refuses, because a silent fallback once caused real damage.

6 · Coherence enforcement — the largest divergence

superpowers

A review refactors in place

A fix is patched directly

A determined change is just coded

the first impl that works lands — even when the repo already has a pattern

→

chris-code

One **application engine**: `coherent-change`

Every determined change routes through it

defend → size → (batch a whole set)

discovery proposes & hands off — it never lands its own patches

superpowers has no step that makes a change **fit the codebase it lands in**. chris-code adds one: discover candidates from the existing code, defend the most coherent against the alternatives (a correctness table + a real rebuttal of each), **size** the work (build inline, or route to spec → plan → execution), and **batch** a whole set of findings into one coordinated change.

What stayed the same

- ✓ **brainstorming** — intent → approaches → design
- ✓ **test-driven-development** — RED-GREEN-REFACTOR
- ✓ **systematic-debugging** — four phases, Iron Law
- ✓ **finishing-a-development-branch** — merge / PR / keep / discard
- ✓ **dispatching-parallel-agents**, **receiving-code-review**, **using-chris-code** (light edits)

Skills you know — what changed most

Where muscle memory will mislead you. Framed before → after:

Skill	superpowers	chris-code
writing-plans	the plan skill: exhaustive, full code per step	slimmed to `lean-plan`; spec → `lean-spec`
subagent-driven-development	2-stage review; parallel discouraged	3 gates/task (review → scripted triage → commit gate), scope dispatch, staged parallelism
verification-before-completion	"what command proves this? run it"	tests → lints, then design review, requirements, spec-blind intent and mutation together as one close round, triaged once (max 2 rounds)
requesting-code-review	primary, mandatory path	demoted to ad-hoc; base `HEAD~1` → `merge-base HEAD main`

What's new, and why

New skills (11)

lean-spec **coherent-change** **remediating-issues** **regression-test** **python-review**
rust-review **technical-review** **bug-hunt** **test-sweep** **code-archaeology** **release**

New agents (15) — the layer superpowers doesn't have

3 coders (`python` / `pytorch` / `rust`) — exclusive, most-specific wins

3 quality-reviewers — additive, dispatched alongside the spec review

2 review-lite gates — idiom + lint over the whole change

2 design-reviewers (`python` / `rust`) — senior cohesion at the verification gate

2 conformance reviewers — `spec-reviewer` (promoted from a prompt template) + `intent-reviewer` (spec-blind, new)

`bug-hunter` — adversarial edge-case tests · `claim-checker` — settles a cannot-verify item by quoting code

`mutation-tester` — polyglot mutation gate in an isolated worktree (Step 6 / on-demand)

The net

chris-code is a **true superset**: every superpowers skill is present (one renamed, one split), plus 11 new skills and a 15-agent layer.

Same spine. Leaner artifacts, mechanical dispatch, harder gates.

Full write-up: `superpowers-comparison.md`